

1. ¿Cómo se complementan la vista lógica y la vista de desarrollo?

Para mí estas dos vistas son muy bueno que esten juntas. La vista lógica define las entidades del sistema, sus responsabilidades y relaciones. La vista de desarrollo muestra cómo eso queda organizado en el código real.

En nuestro proyecto que es una API REST para una Red Nacional de Historias Clínicas, la vista lógica identifica cuatro entidades principales: Hospital, Usuario, Paciente e HistoriaClinica, cada una con un rol claro dentro del sistema. Eso se refleja directamente en la vista de desarrollo donde están todas esas clases con SQLAlchemy, un schemas con los contratos de entrada y salida, y una carpeta con un archivo separado para hospitales, pacientes e historias. La estructura del código espeja la estructura lógica del sistema.

Eso hace que el sistema sea mantenible: si hay que cambiar la lógica de cómo se crean las historias clínicas, se va directo sin tocar nada más. Si ambas vistas no estuvieran alineadas y todo estuviera junto en un solo archivo enorme, escalar el sistema o agregar nuevas funcionalidades se volvería mucho más riesgoso.

2. ¿Qué consecuencias tiene una inconsistencia entre la vista de procesos de negocio y la vista lógica?

Esto es algo que detecté trabajando en el proyecto. El proceso de negocio dice que un hospital debe ser aprobado antes de que sus médicos puedan registrar historias clínicas. Ese flujo de aprobación existe en el negocio.

Ahora bien, si ese proceso de negocio no se hubiera tenido en cuenta al diseñar la vista lógica es decir, si el modelo Hospital no tuviera ese campo, el sistema dejaría registrar historias clínicas desde hospitales que todavía no han sido validados en la red nacional. Eso, en un sistema de salud, es un problema grave: datos de pacientes registrados por instituciones no autorizadas, sin posibilidad de auditoría. En el código hay una validación explícita que verifica ese campo antes de permitir crear una historia. Si la vista lógica no lo soportara, esa regla de negocio simplemente no existiría en el sistema, aunque sí existiera en el proceso.

3. ¿Por qué la vista física no debería dejarse para el final?

En nuestro proyecto, la base de datos es SQLite, lo cual es práctico para desarrollar y hacer pruebas locales. Pero si el sistema se fuera a desplegar de verdad como una red nacional que conecta múltiples hospitales en Colombia, esa decisión de infraestructura cambia todo.

SQLite no soporta múltiples escrituras concurrentes, así que en un entorno real con varios hospitales consultando y registrando al mismo tiempo, el sistema colapsaría. Eso significaría migrar a PostgreSQL o MySQL, lo cual implicaría revisar la configuración de database.py, las cadenas de conexión, y posiblemente algunos comportamientos de los modelos. Además, si el sistema debe estar disponible 24/7 como se esperaría de un sistema de salud nacional, habría que pensar desde el inicio en replicación, balanceo de carga y zonas de disponibilidad en la nube.

El punto es que si esas decisiones se toman tarde, uno termina rediseñando partes del sistema que ya estaban terminadas. Haberlo pensado desde el principio habría evitado ese trabajo doble.